

Django Practical Examination

Restaurant Menu Management System

Duration: 3 Hours

Maximum Marks: 100

Technology: Django, SQLite, HTML, CSS

Type: Individual Practical Examination

1. Problem Statement

A restaurant wants a simple web-based **Menu Management System** to manage the food items available in its restaurant.

The restaurant staff should be able to add new food items, view all available food items, update existing food items, and remove food items from the menu.

In addition to the basic CRUD operations, the system should provide **category-based filtering** and allow staff to quickly change the availability of a food item directly from the menu listing.

You are required to develop this application using **Django**.

2. Functional Requirements

A. Food Item Management — CRUD

Create a Django model named `FoodItem`.

The model should contain the following fields:

Field	Suggested Type	Requirement
Name	CharField	Required

Category	CharField	Required
Price	DecimalField	Required
Description	TextField	Required
Image	ImageField	Optional/Required
Availability	BooleanField	Required
Created At	DateTimeField	Automatically generated

1. Create Food Item

Create a form that allows the restaurant staff to add a new food item.

The form should contain:

- Food Name
- Category
- Price
- Description
- Image
- Availability

After successful submission:

- The food item should be stored in the database.
 - The user should be redirected to the food-item listing page.
 - A suitable success message should be displayed.
-

2. Display Food Items

Create a page that displays all food items.

Each food item should display:

- Image
- Name
- Category
- Price
- Description
- Availability status
- Edit option

- Delete option

Example:

RESTAURANT MENU

[Image]

Chicken Biryani
Category: Main Course
Price: ₹180

Delicious chicken biriyani
Status: Available

[Edit] [Delete] [Make Unavailable]

The listing should be retrieved using the **Django ORM**.

3. Update Food Item

Provide an **Edit** option for every food item.

The user should be able to modify:

- Name
- Category
- Price
- Description
- Image
- Availability

After updating, redirect the user back to the listing page with an appropriate success message.

4. Delete Food Item

Provide a **Delete** option for every food item.

Before deleting, the system should ask the user for confirmation.

Example:

Are you sure you want to delete
"Chicken Biryani"?

[Cancel] [Delete]

After successful deletion, display an appropriate success message.

3. Additional Functionalities

The following functionalities are mandatory and carry additional marks.

5. Category Filter

The restaurant menu should allow users to filter food items based on their category.

Example categories:

- Starters
- Main Course
- Desserts
- Beverages
- Snacks

The listing page should contain a category filter.

Example:

Category:

[All Categories ▼]

[Search]

If the user selects:

Main Course

only food items belonging to the **Main Course** category should be displayed.

Requirements

- **All Categories** should display all food items.
 - Selecting a category should display only matching food items.
 - The filtering should be performed using the **Django ORM**, not by hiding elements using JavaScript.
-

4. Availability Toggle

6. Toggle Food Availability

The restaurant staff should be able to change the availability of a food item directly from the listing page.

For example:

Chicken Biryani
Status: Available

[Mark as Unavailable]

After clicking:

Chicken Biryani
Status: Unavailable

[Mark as Available]

The user should **not need to open the Edit page** to change availability.

Requirements

- If the current value is **True**, change it to **False**.
- If the current value is **False**, change it to **True**.
- Save the updated value in the database.
- Redirect back to the menu page.
- Display an appropriate success message.

Example:

Chicken Biryani marked as unavailable.

5. Validation Requirements

The application should perform basic validation.

Food Name

- Cannot be empty.

Category

- Cannot be empty.

Price

- Must be greater than 0.
- Negative prices should not be accepted.

Description

- Cannot be empty.

Image

- Should accept an appropriate image file.

Validation errors should be displayed clearly to the user.

6. User Interface Requirements

The application should have a clean and understandable interface.

Required Pages

Students should implement at least:

Page 1 — Food Item List

Restaurant Menu

[Add Food Item]

Category: [All ▼]

Food Item Cards / Table

Page 2 — Add Food Item

Add Food Item

Name:

Category:

Price:

Description:

Image:

Availability:

[Save Food Item]

Page 3 — Edit Food Item

Edit Food Item

Name:

Category:

Price:

Description:

Image:

Availability:

[Update Food Item]
